

Project 1: Content-based Recommendation

Due: 2026/09/29, 23:59

프로젝트 목표

Content-based recommendation 알고리즘 구현

Project Environment

- NumPy, Pandas 외 비슷한 종류의 다른 라이브러리를 사용하거나 사용하지 않고 구현해도 무방
 - Python 3.12
 - sentence-transformers
 - NumPy (Optional)
 - Pandas (Optional)

Dataset

- Steam dataset
 - ratings.csv: 사용자가 게임에 대해 평가한 평점 (1~5 사이 정수값)
 - games_metadata.json: 게임의 description 및 tag 정보
 - test.csv (채점용, 제공되지 않음): 배포되는 ratings.csv에는 일부 interaction이 제외되어 있음. 제외된 (user_id, game_id) 쌍의 실제 rating을 기반으로 추천 예측 점수 평가 진행.

Task

1. Description을 이용한 game representation 추출
 - a. "games_metadata.json"에서 게임의 description 추출 (game_id를 기준으로 정렬).
 - b. HuggingFace에서 Sentence Transformers의 'all-MiniLM-L6-v2' 모델 load.
 - c. 모델을 활용하여 description 벡터화.

	0	1	2	3	4	5	6	7	8	9	...	379	380	381	382	383
0	0.001041	-0.045478	-0.062070	-0.076691	0.026874	0.051717	0.055058	0.047182	-0.019669	0.061317	...	-0.004114	0.064459	0.000631	-0.058914	0.026056
1	-0.038457	-0.031913	-0.089972	-0.076248	-0.049347	0.033999	-0.009941	0.043622	-0.023928	0.069134	...	0.060860	0.005809	-0.019929	-0.067986	-0.045055
2	-0.093616	-0.007310	0.010459	-0.095035	0.000026	-0.035315	0.045138	0.027874	0.006711	0.064478	...	0.059040	0.089066	0.070101	0.012698	0.071980
3	-0.091260	0.040785	-0.038663	0.018047	-0.023705	0.056139	0.008081	-0.093186	-0.061737	0.017451	...	-0.053333	0.032827	-0.016889	-0.136893	0.008934
4	-0.007795	-0.015213	-0.029448	-0.004154	-0.038398	-0.025534	0.066754	0.043830	0.008254	0.020648	...	0.010767	0.000413	-0.094760	-0.013317	-0.043102
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
19750	-0.041654	-0.082430	0.000410	0.007684	-0.095860	-0.026088	0.034065	0.044144	0.028075	0.034148	...	-0.004965	0.010128	-0.015821	-0.009476	0.055329
19751	-0.062690	0.005425	-0.053632	0.033136	0.034073	0.072373	0.020005	-0.027583	-0.024508	0.040040	...	0.008005	0.060344	0.056230	-0.090735	0.065827
19752	0.004030	0.045967	0.021388	0.023794	0.010000	-0.047586	-0.031042	-0.098505	-0.107186	-0.028943	...	-0.009221	-0.027316	0.039791	-0.028098	0.054882
19753	-0.008399	0.024680	-0.073492	-0.032865	0.047352	0.020364	-0.013012	-0.017648	0.028694	0.008833	...	-0.032049	-0.067751	0.029393	-0.039685	0.014442
19754	-0.045388	0.022103	0.061770	-0.077471	-0.033709	-0.064304	0.091284	0.007283	-0.039803	0.044738	...	0.085105	0.012297	0.013086	-0.029977	0.003193

그림 1 Semantics-based game representation example, shape=[19755, 384]

2. Tag를 이용한 game representation

a. "games_metadata.json"에서 게임의 unique tags 추출.

i. # of unique tags = 440

b. Tag의 IDF를 계산.

i. IDF 공식: $\text{np.log10}(\text{total_count} / \text{tag_count})$

1. total_count: 전체 game 개수

2. tag_count: 각 tag가 모든 game에 나타난 횟수

c. TF는 한 game에 대해 tag가 몇 번 등록되었는지에 따라 계산.

i. TF 공식: $n(d, t) / n(d)$

d. TF-IDF를 이용하여 game representation 계산.

i. 1에서와 마찬가지로 각 행은 game_id를 기준으로 정렬되어 있어야 함.

ii. 각 열은 tags에 대해서 alphabetical order 기준으로 정렬되어 있어야 함.

	1980s	1990's	2.5D	2D	2D Fighter	2D Platformer	360 Video	3D	3D Fighter	3D Platformer	...	World War I	World War II	Wrestling	Zombies	eSports
0	0.0	0.073906	0.0	0.000000	0.0	0.0	0.0	0.000000	0.0	0.0	...	0.000000	0.000000	0.0	0.0	0.0
1	0.0	0.000000	0.0	0.000000	0.0	0.0	0.0	0.000000	0.0	0.0	...	0.125517	0.088467	0.0	0.0	0.0
2	0.0	0.092382	0.0	0.000000	0.0	0.0	0.0	0.000000	0.0	0.0	...	0.000000	0.000000	0.0	0.0	0.0
3	0.0	0.073906	0.0	0.000000	0.0	0.0	0.0	0.000000	0.0	0.0	...	0.000000	0.000000	0.0	0.0	0.0
4	0.0	0.000000	0.0	0.000000	0.0	0.0	0.0	0.042715	0.0	0.0	...	0.000000	0.000000	0.0	0.0	0.0
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
19750	0.0	0.000000	0.0	0.026363	0.0	0.0	0.0	0.000000	0.0	0.0	...	0.000000	0.000000	0.0	0.0	0.0
19751	0.0	0.000000	0.0	0.000000	0.0	0.0	0.0	0.056953	0.0	0.0	...	0.000000	0.000000	0.0	0.0	0.0
19752	0.0	0.000000	0.0	0.000000	0.0	0.0	0.0	0.050253	0.0	0.0	...	0.000000	0.000000	0.0	0.0	0.0
19753	0.0	0.000000	0.0	0.065908	0.0	0.0	0.0	0.000000	0.0	0.0	...	0.000000	0.000000	0.0	0.0	0.0
19754	0.0	0.000000	0.0	0.000000	0.0	0.0	0.0	0.000000	0.0	0.0	...	0.000000	0.000000	0.0	0.0	0.0

그림 2 TF-IDF-based game representation example, shape=[19755, 440]

3. 두 game 사이의 cosine similarity 계산. (평가 시 matrix를 확인하지 않음)

a. 1과 2에서 만든 representation을 바탕으로 각각 계산.

b. 계산 방법에 따른 0.01% 정도의 차이는 최종 결과에 영향 없음.

	0	1	2	3	4	5	6	7	8	9	...	19750	19751	19752	19753	19754
0	1.000000	0.488360	0.365246	0.300999	0.455248	0.197645	0.350193	0.382077	0.386646	0.383801	...	0.220501	0.309671	0.042861	0.115442	0.041897
1	0.488360	1.000000	0.264465	0.265301	0.295119	0.127530	0.129138	0.216340	0.347474	0.300612	...	0.165441	0.214354	-0.002994	0.086322	-0.012430
2	0.365246	0.264465	1.000000	0.312724	0.343721	0.244655	0.380981	0.367851	0.409528	0.552800	...	0.291129	0.383734	0.162134	0.096891	0.129537
3	0.300999	0.265301	0.312724	1.000000	0.283936	0.399664	0.212174	0.267664	0.247637	0.225525	...	0.215979	0.366552	0.078158	0.222288	0.148865
4	0.455248	0.295119	0.343721	0.283936	1.000000	0.167398	0.229014	0.298859	0.301004	0.294255	...	0.309860	0.323539	0.074186	0.166655	-0.007203
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
19750	0.220501	0.165441	0.291129	0.215979	0.309860	0.068664	0.136356	0.163748	0.150676	0.184109	...	1.000000	0.178490	0.184901	0.306326	0.156163
19751	0.309671	0.214354	0.383734	0.366552	0.323539	0.193588	0.246982	0.299791	0.274798	0.397508	...	0.178490	1.000000	0.060207	0.153279	0.207307
19752	0.042861	-0.002994	0.162134	0.078158	0.074186	0.022985	0.065953	0.147407	0.013781	0.096089	...	0.184901	0.060207	0.999999	0.180326	0.017889
19753	0.115442	0.086322	0.096891	0.222288	0.166655	0.175438	0.074424	0.089913	0.094754	0.045310	...	0.306326	0.153279	0.180326	1.000000	0.051063
19754	0.041897	-0.012430	0.129537	0.148865	-0.007203	0.128908	-0.060784	-0.031050	0.027425	0.041932	...	0.156163	0.207307	0.017889	0.051063	1.000000

그림 3 Semantics-based game-game similarity matrix example, shape=[19755, 19755]

	0	1	2	3	4	5	6	7	8	9	...	19750	19751	19752	19753	19754
0	1.000000	0.430080	0.521191	0.252016	0.295433	0.267137	0.152238	0.311136	0.392615	0.506917	...	0.000000	0.051058	0.022476	0.000000	0.067110
1	0.430080	1.000000	0.354473	0.289662	0.240466	0.230996	0.169454	0.163114	0.718832	0.204295	...	0.001489	0.029861	0.026573	0.022115	0.002964
2	0.521191	0.354473	1.000000	0.435652	0.438688	0.400806	0.243545	0.366021	0.248589	0.522413	...	0.031254	0.034600	0.030842	0.000000	0.098224
3	0.252016	0.289662	0.435652	1.000000	0.243271	0.814631	0.722240	0.796174	0.280908	0.323501	...	0.213665	0.007878	0.029359	0.002875	0.097689
4	0.295433	0.240466	0.438688	0.243271	1.000000	0.297541	0.233178	0.224454	0.167120	0.326680	...	0.075187	0.058450	0.051489	0.000000	0.000000
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
19750	0.000000	0.001489	0.031254	0.213665	0.075187	0.210055	0.212541	0.204589	0.000000	0.121528	...	1.000000	0.003907	0.182011	0.136965	0.011733
19751	0.051058	0.029861	0.034600	0.007878	0.058450	0.007744	0.032817	0.031590	0.195947	0.126040	...	0.003907	1.000000	0.221145	0.006826	0.032987
19752	0.022476	0.026573	0.030842	0.029359	0.051489	0.028863	0.029205	0.028112	0.088021	0.029033	...	0.182011	0.221145	1.000000	0.003118	0.039335
19753	0.000000	0.022115	0.000000	0.002875	0.000000	0.002827	0.002860	0.002753	0.020222	0.002843	...	0.136965	0.006826	0.003118	1.000000	0.005135
19754	0.067110	0.002964	0.098224	0.097689	0.000000	0.010340	0.010463	0.093539	0.103535	0.010401	...	0.011733	0.032987	0.039335	0.005135	1.000000

그림 4 TF-IDF-based game-game similarity matrix example, shape=[19755, 19755]

4. 예측 평점 계산.

a. "ratings.csv"에서 특정 user_id가 평가한 game_id와 rating 추출.

(# of interaction=n)

b. 3에서 만든 두 game-game similarity matrices와 아래 설명을 기반으로 해당 user_id가 전체 game에 남길 것으로 추정되는 점수 계산.

Content 기반 Item-based Collaborative Filtering

i. 특정 user_id가 평가한 game_id와 전체 game 사이의 유사도 추출.

- game-game similarity matrix에서 추출.

- shape=[n, 19755] where n = # of interacted games

ii. 특정 user_id가 평가한 각 game_id에 대해 남긴 rating 추출.

- shape=[n, 1]

iii. 특정 user_id가 평가한 각 game_id의 평점과 각 game_id와 전체 game 간의 유사도를 활용해 가중치를 반영한 예측 평점 추출.

- shape=[19755, 1]

iv. 특정 user_id가 평가한 game_id들과 전체 game 간의 유사도의 합을 활용하여 정규화.

- shape=[19755, 1]

- Hint: i. 에서 구한 유사도를 활용, 유사도의 합이 0이 될 때를 방지하기 위해 매우 작은 수(eps) 포함

- c. 4.b에서 계산한 Semantics-based 및 TF-IDF-based 추정 점수를 가중합하여 최종 추정 점수 산출.
 - i. $\text{Score}_{\text{weighted}} = \alpha \cdot \text{Score}_{\text{Semantics-based}} + (1 - \alpha) \cdot \text{Score}_{\text{TF-IDF-based}}$
 - ii. $0 < \alpha < 1$
 - iii. 0.5, 0.5가 아닌 최적의 값을 자유롭게 도출 (0.5, 0.5를 적용한 값보다 성능이 좋아야 함, Hint. ratings.csv에서 일부를 validation으로 나눠 확인할 것)
 - iv. 가중치 최적값 및 도출한 방법을 report에 명시.

평가기준

1. 4-b 특정 user_id의 게임 추천 리스트의 상위 20개 대한 user_id, game_id, prediction_score가 정답과 일치하는지 평가
 - a. user_id에 따른 game_id 추천 순서.
 - b. 추천 시, user가 ratings.csv에서 interact하지 않은 games만 추천하여야 함.
2. 4-c 특정 user_id, game_id에 대해 추정한 prediction_score에 대해 평가
 - a. Testset에서의 실제 rating_score와 prediction_score의 RMSE
3. Output format 준수 여부 평가.
 - a. main.py 실행 시 results 폴더가 생성되어야 함.
 - b. main.py 실행 시 tfidf_output.txt, semantics_output.txt, score_prediction_output.txt 파일이 results 폴더 내에 생성되어야 함.
 - c. 4-b (input_recommendation.txt)
 - i. 두 가지의 알고리즘(TF-IDF-based, Semantics-based)에 따라 user_id당 순서대로 20개씩 출력.
 - ii. user_id당 prediction_score 기준으로 내림차순 정렬하여 user_id, game_id와 prediction_score 차례대로 출력.
 1. prediction_score가 같을 경우, game_id를 기준으로 오름차순 정렬.
 - d. 4-c (input_score_prediction.txt)
 - i. user_id, game_id를 읽어서 아래 format에 맞게 prediction_score를 출력해야 함.
 - e. 필드(user_id, game_id, prediction_score) 간 구분자는 공백 없이 콤마(,)를 사용.
 - f. prediction_score의 경우, 소수점 아래 4번째까지 남기고 반올림.

Example

```
# input_recommendation.txt format (4-b)
# 한 줄에 한 개의 user_id, 오름차순으로 정렬.
1
33
100
250
3402
```

```
# {tfidf, semantics}_output.txt format
# 상위 20개씩 출력, comma(',')로 구분.
# 순서대로 user_id, game_id, prediction_score
# user_id당 prediction_score를 ordering 순서대로 출력, score가 같으면 game_id에 대
해서 오름차순 정렬.
# 현재 예시에서는 5 * 20 = 100 줄이 출력되어야 함.
# 아래 출력 결과는 출력 형태 예시일 뿐, 실제 정답과는 무관함.
1,31,4.3331
1,35,4.2213
1,20,4.2202
1,2045,4.1125
...
3402,20,4.6453
...
```

```
# input_score_prediction.txt format (4-c)
# 한 줄에 한 개의 user_id, game_id
1;35
3402;20
...
```

```
# score_prediction_output.txt format
# 순서대로 user_id, game_id, prediction_score, 출력 형태 예시
1,35,4.3331
3402,20,3.5513
...
...
```

제출

제출방법

- ETL 과제 탭으로 제출.
- Source code와 Report를 함께 zip 파일로 묶어서 첨부.
- 사용하는 라이브러리는 requirements.txt에 모두 명시할 것 (설치 코드를 main.py에 포함하지 말 것)
- 제목 양식은 다음을 준수: PRJ1_202600000_홍길동.zip

```
# 제출 상태(구현 파일에 대한 이름 및 파일 개수 제한 없음)
PRJ1_202600000_홍길동.zip
├─ PRJ1_202600000_홍길동.pdf
├─ main.py (코드 구현)
├─ requirements.txt (필요한 라이브러리 명시)
├─ data
│   ├── games_metadata.json
│   └── ratings.csv
├─ results (제출 x, 채점 시 생성)
│   ├── tfidf_output.txt (제출 x, 채점 시 생성)
│   ├── semantics_output.txt (제출 x, 채점 시 생성)
│   └── score_prediction_output.txt (제출 x, 채점 시 생성)
├─ input_recommendation.txt (제출 x, 채점 시 위치)
└─ input_score_prediction.txt (제출 x, 채점 시 위치)
```

- 지연 제출

- 제출 기한 이후 24시간 이내 제출 시 10% 감점.
- 제출 기한 이후 24시간 이후, 48시간 이내 제출 시 20% 감점.
- 제출 기한 48시간 이후에는 점수 없음.
- 부정 행위는 0점 처리.
 - 다른 수강생의 코드를 참조하는 행위.
 - 이전에 수강한 사람의 코드를 참조하는 행위.
 - 제출한 소스코드에 대해 표절 방지 프로그램을 돌릴 예정.
- 본 문서에 명시되어 있는 제출 및 출력 양식을 지키지 않을 시 감점.

Source Code

- 제공된 main.py를 기반으로 구현.
 - 주석에 학번 이름 작성

Report

- 각 문제별 코드 설명, 어려웠던 점 및 해결 방법, 느낀 점 등의 내용을 포함하게 작성.
- 길이는 최대 A4 2장까지, PDF 형식으로 제출.
- 파일 제목은 다음 양식을 준수: PRJ1_202600000_홍길동.pdf